

Agentes CLI na Prática

Do conceito à orquestração de múltiplos agentes em produção

Minicurso — TADS / IFRN

Material completo · 6 módulos · exercícios & quizzes



Agenda

0:00–0:10

Abertura & Contexto

0:10–0:30

O que são Agentes CLI

0:30–0:55

Harnesses, Skills & Extensões

0:55–1:10

Economia de Contexto & PRDs

1:10–1:30

Prática: Multi-agentes em Python

Bônus

Deploy, CI/CD & Agentes em Produção

① Abertura & Contexto 0:00 – 0:10 min

"Você já passou horas procurando um bug, lendo docs, abrindo 30 abas no browser? Imagine um assistente que lê o projeto, entende o erro, pesquisa a solução e abre o PR — enquanto você toma um café."

Por que isso importa AGORA (2025)?

Em 2023 LLMs só conversavam. Em 2024 ganharam "tool use". Em 2025 viraram agentes autônomos que rodam no seu terminal, leem seu código, executam comandos e abrem Pull Requests. A diferença entre um dev que sabe orquestrar agentes e um que não sabe está virando o novo "saber usar Google" — uma habilidade que multiplica produtividade em 3-10x. Este minicurso te dá o mapa: do conceito à orquestração de múltiplos agentes em produção.

O que o aluno vai sair sabendo fazer:

- Usar agentes CLI (Claude Code, Gemini CLI) para resolver problemas reais
- Entender o loop ReAct e a anatomia de uma chamada de ferramenta (tool call)
- Configurar harnesses de browser, ferramentas (MCP) e skills customizadas
- Economizar tokens e compactar contexto em projetos longos
- Escrever PRDs que um agente transforma em código funcional
- Orquestrar múltiplos agentes em Python com CrewAI / LangGraph / AutoGen

O ecossistema em 1 minuto:

Gemini CLI — CLI do Google, gratuito, bom ponto de entrada

Claude Code — Agente de engenharia da Anthropic (você está usando agora)

Cursor / Windsurf — IDEs com agentes embutidos no editor

Harness — Adaptadores que dão ao agente browser, terminal, arquivos

MCP (Model Context Protocol) — Padrão aberto para conectar agentes a ferramentas

Skills / SKILL.md — Instruções especializadas carregadas sob demanda

Pré-requisitos para acompanhar

Python 3.10+, Node 18+, uma conta com API key (Anthropic, Google ou OpenAI) e um terminal. Não precisa ser expert — se você já fez um CRUD, está pronto. Tudo que mostrarmos roda em máquina comum de estudante.

Mini-glossário (volte aqui quando travar):

LLM — Large Language Model — o 'cérebro' (ex: Claude, Gemini, GPT)

Agente — LLM + ferramentas + loop de decisão autônomo

Tool / Ferramenta — Função que o agente pode chamar (ler arquivo, rodar bash...)

Contexto — Tudo que o modelo 'enxerga' numa requisição (texto + código)

Token — Pedaco de texto (~4 caracteres). É a unidade de custo

Harness — A 'casca' que conecta o LLM ao mundo real (terminal, browser)

MCP — Protocolo para plugar ferramentas externas no agente

PRD — Product Requirements Document — o briefing do que construir

Aquecimento 📌 Exercício 0 — Instale e fale com um agente

Instale o Gemini CLI (gratuito) OU o Claude Code e peça a ele a primeira tarefa:

"Liste os arquivos desta pasta e me diga em uma frase o que este projeto faz."

Observe que ele NÃO só responde — ele executa comandos para descobrir a resposta.

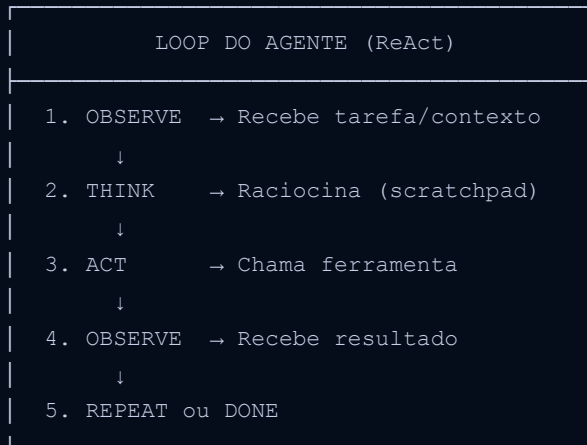
💡 Gemini CLI: `npm install -g @google/gemini-cli && gemini`

💡 Claude Code: `npm install -g @anthropic-ai/claude-code && claude`

💡 Repare quantas vezes o agente 'pensa → age → observa' antes de responder

② O que são Agentes CLI 0:10 – 0:30 min

Loop de raciocínio do agente (ReAct = Reason + Act):



Anatomia de um agente (as 4 peças)

1. MODELO (LLM): o cérebro que raciocina e decide.
2. FERRAMENTAS (tools): mãos e olhos — ler/escrever arquivos, rodar bash, pesquisar web.
3. LOOP / ORQUESTADOR: o código que chama o modelo, executa as ferramentas e devolve os resultados — repetindo até a tarefa terminar.
4. CONTEXTO / MEMÓRIA: o histórico da conversa + arquivos relevantes que o modelo enxerga.

Tire qualquer peça e deixa de ser agente: sem ferramentas é só um chatbot; sem loop é uma única resposta; sem contexto ele esquece tudo.

Chatbot comum vs Agente CLI:

Chatbot comum	Agente CLI
Só responde com texto	Executa ações reais
Sem memória de arquivos	Lê e edita seu projeto
Não acessa internet	Pode pesquisar, abrir URLs
Uma resposta por vez	Loop até concluir a tarefa
Você implementa	Ele implementa por você
Você verifica tudo	Ele roda os testes e se corrige

Como o modelo 'pede' uma ferramenta — o tool call por dentro:

```
# O modelo NÃO roda código. Ele devolve um JSON pedindo uma ação.
# O HARNESS executa e devolve o resultado. Exemplo de um turno:

# 1) O modelo responde com um pedido de ferramenta:
{
  "type": "tool_use",
  "name": "bash",
  "input": { "command": "pytest -q" }
}

# 2) O harness roda 'pytest -q' e devolve o resultado ao modelo:
{
  "type": "tool_result",
  "content": "3 failed, 12 passed in 1.2s"
}

# 3) O modelo lê o erro, decide corrigir, e pede outra ferramenta:
{
  "type": "tool_use",
  "name": "edit_file",
  "input": { "path": "src/auth.py", "old": "<=", "new": "<" }
}

# ...e o loop continua até "0 failed".
```

Exemplo real — pedindo ao agente para criar uma API:

```
# No terminal, com Claude Code:
$ claude "Crie uma API REST em FastAPI com CRUD de usuários,
  usando SQLite, com auth JWT e testes com pytest.
  Siga o padrão do projeto em ./src"

# O agente vai:
# 1. Ler os arquivos em ./src para entender o padrão
# 2. Criar models, routers, schemas
# 3. Escrever os testes
# 4. Rodar pytest e corrigir erros automaticamente
# 5. Retornar com tudo funcionando
```

Ferramentas que um agente CLI tipicamente tem:

- bash / shell — executa comandos no terminal
- read_file / write_file / edit_file — lê e edita arquivos
- glob / grep — encontra arquivos e padrões no código
- web_search / web_fetch — pesquisa e lê páginas da internet
- browser — navega, clica, preenche formulários, tira screenshots
- code_execution — roda Python, Node, etc. num sandbox
- memory — persiste informações entre sessões
- subagent / task — delega sub-tarefas a outros agentes

Níveis de autonomia (e quando usar cada um):

Nível	Comportamento	Quando usar
Sugestão	Mostra o diff, você aplica	Código crítico, produção
Aprovação	Pede OK antes de cada ação	Aprendendo a confiar
Auto (edição)	Edita arquivos sozinho, não roda comandos perigosos	Dia a dia
Auto total (YOLO)	Roda tudo sem perguntar	Sandbox / experimentos descartáveis

Cuidado: o agente é um estagiário brilhante, não um sênior

Ele pode 'alucinar' uma biblioteca que não existe, deletar arquivos errados ou commitar segredos. Regras de ouro: trabalhe em git (dá pra reverter), revise diffs, nunca dê acesso de produção sem necessidade e desconfie de confiança excessiva.

Quiz relâmpago

O que diferencia um agente CLI de um chatbot comum?

☐ O agente usa um modelo maior

☒ O agente executa ações em loop usando ferramentas até concluir a tarefa

☐ O agente é sempre mais barato

☐ O agente não precisa de internet

 A essência é o LOOP + FERRAMENTAS: observar, raciocinar, agir e repetir até terminar — não só responder texto.

Básico **Exercício 1 — Faça o agente trabalhar em loop**

Peça a um agente CLI: "Crie um script Python que conte palavras de um arquivo .txt, escreva um teste com pytest, rode o teste e corrija até passar." Observe e anote: quantas iterações do loop ReAct ele fez? Em qual passo ele se corrigiu sozinho?



Use `--verbose` (ou modo equivalente) para ver o raciocínio



Crie um arquivo .txt de exemplo antes para ele ter o que testar



Se ele acertar de primeira, peça um requisito a mais para forçar uma correção

③ Harnesses, Skills & Extensões 0:30 – 0:55 min

O que é um Harness?

Um harness é o "colete de ferramentas" do agente. Ele define quais capacidades o agente tem acesso naquela sessão e **MEDIA** tudo que ele faz: executa as ferramentas, aplica permissões, faz o sandboxing e devolve os resultados ao modelo.

O Browser Harness, por exemplo, dá ao agente um Chromium real que ele controla via código — ele vê a página, clica em elementos, extrai dados, preenche formulários.

Browser Harness em ação — scraping com agente:

```
# O agente recebe as ferramentas "navigate", "snapshot" e "click"
# e raciocina sobre o que vê na tela
```

```
Tarefa: "Acesse o portal do IFRN, encontre o calendário acadêmico
        de 2025 e extraia todas as datas de provas."
```

```
# Agente pensa:
# → navigate("https://ifrn.edu.br")
# → snapshot() (lê a árvore de acessibilidade da página)
# → encontra o link "Calendário Acadêmico", click(ref)
# → baixa/lê o PDF
# → extrai as datas e retorna em JSON estruturado
```

MCP — o 'USB-C' dos agentes

MCP (Model Context Protocol) é um padrão aberto criado pela Anthropic e adotado amplamente. Em vez de cada agente ter integrações próprias, você roda um "MCP server" (GitHub, Postgres, Slack, Figma...) e **QUALQUER** agente compatível pode plugar nele. É a diferença entre cada aparelho ter um carregador exclusivo e todos usarem USB-C.

Configurando um MCP server (ex: GitHub + Postgres):

```
// .mcp.json (ou config do Claude Code / Cursor)
{
  "mcpServers": {
    "github": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": { "GITHUB_TOKEN": "ghp_xxx" }
    },
    "postgres": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-postgres",
        "postgresql://localhost/ifrn_tads"]
    }
  }
}

// Agora o agente pode: "Liste as PRs abertas e cruze com a tabela
// 'alunos' no banco para ver quem ainda não entregou o trabalho."
```

Escrevendo uma ferramenta customizada (Python / Anthropic SDK):

```
# Toda ferramenta = um schema (o que o modelo vê) + uma função (o que roda)

ferramenta = {
  "name": "consultar_nota",
  "description": "Consulta a nota de um aluno por matricula no SUAP.",
  "input_schema": {
    "type": "object",
    "properties": {
      "matricula": {"type": "string", "description": "Matricula IFRN"}
    },
    "required": ["matricula"]
  }
}

def consultar_nota(matricula: str) -> str:
  # aqui voce chama a API real do SUAP / banco
  return f"Aluno {matricula}: media 8.5, frequencia 92%"

# O harness liga o schema a funcao: quando o modelo pede 'consultar_nota',
# o seu codigo roda e o resultado volta pro modelo.
```

🧩 O que é uma SKILL.md?

```
---
name: fastapi-padrao-ifrn
description: Use quando criar APIs FastAPI no projeto IFRN.
             Contém convenções, estrutura de pastas e padrões.
---

# Padrão FastAPI — IFRN TADS

## Estrutura de pastas
src/
  routers/      # Um arquivo por recurso
  models/       # SQLAlchemy models
  schemas/      # Pydantic schemas
  services/     # Lógica de negócio

## Convenções
- Sempre usar async/await
- Nomear rotas em português: /usuarios, /disciplinas
- Testes obrigatórios em tests/ com pytest
```

A SKILL é injetada no contexto SÓ quando relevante (carregamento sob demanda). Economiza tokens e garante que o agente siga seus padrões sem você repetir a cada conversa.

Técnicas para estender o agente (da mais simples à mais avançada):

- CLAUDE.md / GEMINI.md — memória fixa do projeto, lida em toda sessão
- SKILL.md files — padrões carregados sob demanda quando relevantes
- Slash commands customizados — atalhos para prompts que você repete
- MCP Servers — plugue GitHub, Jira, Slack, banco de dados via protocolo padrão
- Custom tools — funções Python/JS que o agente pode chamar
- Hooks — código que roda ANTES/DEPOIS de cada ação (lint, format, guard-rails)
- Subagents — agentes especializados que o principal invoca para sub-tarefas

CLAUDE.md — configuração e memória do projeto:

```
# CLAUDE.md (na raiz do projeto)

## Projeto
Sistema de gestão acadêmica do IFRN — TADS 2025

## Stack
- Backend: FastAPI + SQLAlchemy + PostgreSQL
- Frontend: React + Tailwind
- Testes: pytest (backend), Vitest (frontend)

## Regras
- NUNCA commitar na branch main diretamente
- Sempre rodar os testes antes de declarar a tarefa completa
- Usar Português para nomes de variáveis de domínio
- Documentar funções públicas com docstrings

## Contexto
Pasta /docs contém os requisitos funcionais.
Pasta /infra contém os Dockerfiles.
```

Hooks = guarda-corpos automáticos

Um hook pode rodar 'prettier' depois que o agente edita um arquivo, bloquear 'rm -rf', ou rejeitar um commit com segredos. É como ter um revisor automático que nunca dorme. No Claude Code: configure em settings.json (PreToolUse / PostToolUse).

Intermediário 📝 Exercício 2 — Escreva sua primeira SKILL.md

Crie uma SKILL.md que ensine o agente o padrão de commits do seu time (ex: Conventional Commits). Depois peça a ele para commitar uma mudança e veja se ele seguiu o padrão SEM você precisar explicar de novo.

💡 Frontmatter precisa de 'name' e 'description' — a description é o gatilho

💡 Seja específico: dê exemplos de bons e maus commits dentro da skill

💡 Teste: peça um commit ANTES e DEPOIS de adicionar a skill, compare

Intermediário **Exercício 3 — Plugue um MCP server**

Configure o MCP server do GitHub (ou do filesystem) e peça ao agente:

"Resuma as 3 últimas issues abertas e proponha qual atacar primeiro."

Observe que ele agora acessa dados externos sem você colar nada manualmente.

 Comece pelo server-filesystem ou server-github (mais fáceis)

 Precisa de um token? Gere um Personal Access Token com escopo mínimo

 Se não tiver repo, use o server-everything (demo) do MCP para testar

④ Economia de Contexto & PRDs 0:55 – 1:10 min

Por que contexto importa?

Agentes CLI têm uma "janela de contexto" — um limite de tokens que conseguem processar de uma vez (ex: 200K tokens $\approx$ 500 páginas). Quanto mais código, histórico e docs você empurra, mais CARO, mais LENTO e — surpresa — menos PRECISO o agente fica: ele se perde no meio de muito texto ("lost in the middle"). Compactar = economizar dinheiro E melhorar a qualidade das respostas.

A matemática do token (por que você deveria se importar):

```
# Regra de bolso: 1 token  $\approx$  4 caracteres  $\approx$  0,75 palavra (em inglês).
# Português gasta um pouco mais de tokens que inglês.

# Exemplo de custo de UMA sessão longa mal gerenciada:
#   - node_modules colado por engano ..... ~50.000 tokens
#   - histórico de 2h de conversa ..... ~80.000 tokens
#   - 10 arquivos relidos 5x cada ..... ~60.000 tokens
#   TOTAL desperdiçado ..... ~190.000 tokens

# Cada requisição reenvia TODO o contexto. Numa conversa de 40 turnos,
# 190K tokens podem ser reenviados dezenas de vezes = $$$ e lentidão.
# Compactar para ~20K mantém o agente rápido, barato e focado.
```

Técnicas de economia de contexto:

- /compact ou /summarize — pede ao agente para resumir o histórico e continuar
- SKILL.md curtas e precisas — só o essencial, carregadas sob demanda
- .gitignore / .agentignore — esconda node_modules, .venv, builds, datasets
- Sessões por tarefa — comece uma conversa nova para cada feature
- Checkpoints — peça ao agente para escrever um resumo em arquivo e recomeçar
- Subagents — delegue buscas grandes; o subagente devolve só a conclusão
- Docstrings como contexto — código bem documentado = menos explicação necessária
- Aponte arquivos específicos (@arquivo.py) em vez de mandar ler a pasta toda

Compactação manual de contexto (checkpoint em arquivo):

```
# Quando a conversa está longa, peça:

"Resuma o que fizemos até agora em um parágrafo técnico conciso,
incluindo: arquivos criados/modificados, decisões tomadas,
e próximos passos pendentes. Salve em .claude/sessao-2025-06-17.md"

# Depois inicie nova sessão (contexto limpo) com:
"Leia .claude/sessao-2025-06-17.md e continue de onde paramos."
```

Onde colocar a informação (contexto vs RAG vs ferramenta):

Estratégia	Quando usar	Custo de contexto
Colar no prompt	Pouca info, usada agora	Alto (sempre reenviado)
CLAUDE.md / SKILL	Padrões do projeto	Médio (sob demanda)
RAG / busca	Muitos docs, busca pontual	Baixo (só o trecho achado)
Ferramenta/MCP	Dado vivo (banco, API)	Baixo (busca quando precisa)

O que é um PRD e por que é poderoso com agentes?

PRD = Product Requirements Document. Com agentes CLI, um PRD bem escrito substitui horas de explicação. O agente lê o PRD e entrega o projeto. É o briefing que transforma "faz um sisteminha aí" em software de verdade.

```
# PRD: Sistema de Controle de Frequência – IFRN TADS

## Objetivo
Aplicação web para professores registrarem frequência de alunos
em tempo real via QR Code gerado na aula.

## Usuários
- Professor: gera QR, vê relatório de presença
- Aluno: escaneia QR com o celular para registrar presença

## Funcionalidades
### MVP (entregar primeiro)
- [ ] Auth com matrícula IFRN (OAuth ou login simples)
- [ ] Gerar QR Code único por aula (expira em 15 min)
- [ ] Aluno escaneia e registra presença
- [ ] Dashboard do professor com lista de presentes

### Fase 2
- [ ] Exportar para Excel
- [ ] Integração com SUAP

## Stack decidida
- Backend: FastAPI + PostgreSQL
- Frontend: React + Tailwind
- Deploy: Docker no servidor do IFRN

## Critérios de aceite
- QR deve ser único e expirar em 15 minutos
- Não deve ser possível registrar presença duas vezes
- Dashboard atualiza em tempo real (WebSocket ou polling 10s)

## O que NÃO está no escopo
- App mobile nativo
- Biometria
- Integração com câmera
```

Estrutura de um PRD eficiente para agentes:

- Objetivo claro em 1-2 frases (o agente precisa saber o 'porquê')
- Usuários e seus papéis (define regras de negócio)
- Funcionalidades separadas por MVP e fases futuras
- Stack decidida (evita o agente escolher por conta própria)
- Critérios de aceite mensuráveis (o agente sabe quando terminar)
- Fora do escopo explícito (evita que o agente invente features)
- Restrições técnicas (versões, limites, padrões obrigatórios)

Padrões de prompt que funcionam (leve pra vida):

- Dê um PAPEL: 'Você é um arquiteto sênior de FastAPI...'
- Peça um PLANO antes do código: 'Antes de codar, liste os passos e espere meu OK'
- Defina o FORMATO de saída: 'Responda só com o diff' / 'em JSON'
- Dê EXEMPLOS (few-shot): mostre 1 caso de entrada→saída esperada
- Estabeleça CRITÉRIO DE PRONTO: 'só termine quando pytest passar 100%'
- Use NEGATIVOS: 'NÃO instale libs novas sem perguntar'

Intermediário

🔗 Exercício 4 — Escreva um PRD e deixe o agente construir

Escolha um mini-projeto (encurtador de URL, lista de tarefas, bot de avisos da turma). Escreva um PRD seguindo a estrutura acima. Entregue SÓ o PRD ao agente e peça: "Leia o PRD, faça um plano, espere meu OK, depois implemente o MVP com testes." Compare a qualidade com um pedido vago do tipo "faz um app de tarefas".

💡 Force o 'espere meu OK' — você revisa o plano antes de gastar tokens

💡 Inclua a seção 'Fora do escopo' — é o que mais evita retrabalho

💡 Critérios de aceite mensuráveis = o agente sabe quando parar

Básico

🔗 Exercício 5 — Pratique a compactação

Numa conversa longa com um agente, peça um resumo-checkpoint em arquivo, abra uma sessão nova, mande ele ler o arquivo e continuar. Meça (aproximadamente) quanto contexto você economizou comparando o tamanho do resumo com o histórico original.

💡 Peça o resumo em tópicos: feito / decisões / pendências

💡 Bom resumo cabe em < 1 página e ainda permite continuar sem perdas

⑤ Prática: Frameworks Multi-Agentes em Python 1:10 – 1:30 min

Por que multi-agentes?

Um único agente tem limitações de contexto e de especialização. Com múltiplos agentes, você tem um orquestrador que delega para especialistas: um agente pesquisa, outro escreve código, outro revisa, outro testa. Como uma equipe real de desenvolvimento — cada um com seu próprio contexto focado, o que reduz erros e custo.

Mas primeiro: você **PRECISA** de multi-agentes?

Regra honesta: 80% das tarefas um agente único resolve. Multi-agentes adiciona complexidade, custo (mais chamadas de LLM) e pontos de falha. Use quando a tarefa tem etapas **CLARAMENTE** separáveis e especializadas. Não use multi-agente por status.

Principais frameworks:

Framework	Paradigma	Melhor para
CrewAI	Papéis (Crew + Agents + Tasks)	Workflows colaborativos simples
LangGraph	Grafo de estados	Fluxos complexos com condicionais/loops
AutoGen (Microsoft)	Conversas entre agentes	Código colaborativo, debugging
Pydantic AI	Type-safe, Pythonico	Produção, integração com APIs
OpenAI Agents SDK	Handoffs entre agentes	Pipelines com handoff e guard-rails

Padrões de orquestração:

Padrão	Como funciona	Exemplo
Sequencial	A → B → C em fila	Pesquisar → escrever → revisar
Hierárquico	Um gerente delega aos demais	Gerente distribui sub-tarefas
Paralelo	Vários ao mesmo tempo, junta no fim	3 agentes analisam 3 módulos
Loop/condicional	Repete até critério (ex: testes ok)	Gerar→testar→corrigir

CrewAI — equipe de desenvolvimento:

```
# pip install crewai crewai-tools

from crewai import Agent, Task, Crew, Process
from crewai_tools import SerperDevTool, FileReadTool

# Ferramentas disponíveis
search_tool = SerperDevTool()
file_tool = FileReadTool()

# 1. Definir os agentes (papéis)
arquiteto = Agent(
    role="Arquiteto de Software",
    goal="Projetar a estrutura do sistema baseada nos requisitos",
    backstory="""Você é um arquiteto sênior especialista em FastAPI
e sistemas distribuídos. Sempre considera escalabilidade.""",
    tools=[file_tool],
    verbose=True
)

desenvolvedor = Agent(
    role="Desenvolvedor Python",
    goal="Implementar o código limpo e com testes",
    backstory="""Você escreve código Python idiomático, sempre com
type hints, docstrings e cobertura de testes acima de 80%. """,
    verbose=True
)

revisor = Agent(
    role="Code Reviewer",
    goal="Revisar código em busca de bugs, segurança e boas práticas",
    backstory="Você tem olho clínico para vulnerabilidades e code smells.",
    verbose=True
)

# 2. Definir as tarefas
tarefa_arquitetura = Task(
    description="""Leia o PRD em docs/prd.md e projete a arquitetura
do sistema. Entregue: estrutura de pastas, lista de endpoints,
modelos de dados e decisões técnicas.""",
    expected_output="Documento de arquitetura em Markdown",
    agent=arquiteto
)

tarefa_codigo = Task(
    description="""Com base na arquitetura definida, implemente o MVP.
Crie os arquivos em src/ seguindo o padrão FastAPI.""",
    expected_output="Código funcional com testes passando",
    agent=desenvolvedor,
    context=[tarefa_arquitetura] # depende da tarefa anterior
)
```

```
tarefa_revisao = Task(
    description="Revise o código produzido e aponte problemas críticos.",
    expected_output="Relatório de revisão com issues e sugestões",
    agent=revisor,
    context=[tarefa_codigo]
)

# 3. Criar e executar a crew
crew = Crew(
    agents=[arquiteto, desenvolvedor, revisor],
    tasks=[tarefa_arquitetura, tarefa_codigo, tarefa_revisao],
    process=Process.sequential, # ou Process.hierarchical
    verbose=True
)

resultado = crew.kickoff()
print(resultado)
```

LangGraph — fluxo com condicionais (loop de auto-correção):

```
# pip install langgraph langchain-anthropic

from langgraph.graph import StateGraph, END
from langchain_anthropic import ChatAnthropic
from typing import TypedDict

# Estado compartilhado entre os nós
class EstadoProjeto(TypedDict):
    tarefa: str
    codigo: str
    testes_passando: bool
    tentativas: int

llm = ChatAnthropic(model="claude-sonnet-4-6")

# Nós do grafo (cada um é um "agente")
def gerar_codigo(state):
    resposta = llm.invoke(f"Escreva uma função Python que {state['tarefa']}")
    return {"codigo": resposta.content, "tentativas": state["tentativas"] + 1}

def rodar_testes(state):
    # Aqui você rodaria pytest de verdade
    passou = "def test_" in state["codigo"] or "assert" in state["codigo"]
    return {"testes_passando": passou}

def corrigir_codigo(state):
    resposta = llm.invoke(
        "Este código falhou nos testes. Corrija:\n" + state["codigo"]
    )
    return {"codigo": resposta.content}

# Roteador - decide o próximo passo
def decidir_proximo(state):
    if state["testes_passando"]:
        return "finalizar"
    elif state["tentativas"] >= 3:
        return "finalizar"          # desiste após 3 tentativas
    return "corrigir"

# Construir o grafo
grafo = StateGraph(EstadoProjeto)
grafo.add_node("gerar", gerar_codigo)
grafo.add_node("testar", rodar_testes)
grafo.add_node("corrigir", corrigir_codigo)

grafo.set_entry_point("gerar")
grafo.add_edge("gerar", "testar")
grafo.add_conditional_edges(
    "testar", decidir_proximo,
    {"finalizar": END, "corrigir": "corrigir"}
)
```

```

grafo.add_edge("corrigir", "testar")

app = grafo.compile()
resultado = app.invoke({
    "tarefa": "calcule o fatorial de um número",
    "codigo": "", "testes_passando": False, "tentativas": 0
})
print(resultado["codigo"])

```

AutoGen — dois agentes conversando (dev + crítico):

```

# pip install pyautogen

from autogen import AssistantAgent, UserProxyAgent

config = {"model": "gpt-4o", "api_key": "sk-..."}

# Agente que escreve código
dev = AssistantAgent(
    name="dev",
    llm_config={"config_list": [config]},
    system_message="Você escreve Python limpo e com testes."
)

# Agente que executa o código e devolve o resultado/erro
executor = UserProxyAgent(
    name="executor",
    human_input_mode="NEVER",          # 100% automático
    code_execution_config={"work_dir": "saida", "use_docker": False},
    max_consecutive_auto_reply=5
)

# Eles conversam até a tarefa terminar
executor.initiate_chat(
    dev,
    message="Implemente bubble sort, rode um teste e mostre a saída."
)

```

Indo para produção — o que ninguém te conta:

- Custo: cada agente extra = mais chamadas de LLM. Meça tokens por execução
- Loops infinitos: SEMPRE coloque max_tentativas / max_turns
- Observabilidade: use LangSmith / logs para ver o que cada agente fez
- Determinismo: temperatura baixa + prompts claros = menos surpresas
- Avaliação: crie um conjunto de testes (evals) para medir se está melhorando
- Falhas: trate timeouts e respostas inválidas — agentes erram, planeje pra isso
- Segurança: sandbox para execução de código, nunca rode código não confiável solto

Usando CrewAI OU LangGraph, monte uma crew de 3 papéis (pesquisador, dev, revisor) que receba o PRD que você escreveu no Exercício 4 e produza: um doc de arquitetura, o código do MVP e um relatório de revisão. Adicione um limite de tentativas e logue o custo aproximado em tokens.

💡 Comece com `Process.sequential` — mais fácil de depurar

💡 Reaproveite o PRD do Ex.4 como entrada (context da primeira task)

💡 Adicione um critério de pronto: revisor aprova ou manda corrigir (loop)

💡 Veja `exemplos-praticos-agentes.py` e `exercicios.py` para o esqueleto pronto

Dicas finais para levar pra vida:

- Comece simples — um agente único já resolve 80% dos casos
- CLAUDE.md / GEMINI.md são seus melhores amigos: documente tudo
- Escreva PRDs antes de código — o agente entrega muito mais com um bom briefing
- Compacte o contexto regularmente em sessões longas
- Use multi-agentes quando a tarefa tiver etapas claramente separáveis
- Revise sempre o que o agente produziu — você é o engenheiro, ele é o estagiário

Documentação para se aprofundar:

- **Claude Code Docs** — Guia oficial do agente da Anthropic
<https://docs.anthropic.com/en/docs/claude-code>
- **Gemini CLI** — CLI gratuito do Google (open source)
<https://github.com/google-gemini/gemini-cli>
- **Model Context Protocol** — Especificação e servers MCP
<https://modelcontextprotocol.io>
- **CrewAI Docs** — Multi-agentes por papéis
<https://docs.crewai.com>
- **LangGraph Docs** — Grafos de estado para agentes
<https://langchain-ai.github.io/langgraph/>
- **AutoGen** — Conversas multi-agente (Microsoft)
<https://microsoft.github.io/autogen/>
- **Anthropic — Building Agents** — Padrões de design de agentes
<https://www.anthropic.com/research/building-effective-agents>
- **Prompt Engineering Guide** — Técnicas de prompt (ReAct, few-shot...)
<https://www.promptingguide.ai>

⑥ Bônus: Deploy, CI/CD & Agentes em Produção Material extra

Do terminal para a nuvem: onde agentes rodam

Até agora você rodou o agente na sua máquina, interativamente. Em produção ele roda SOZINHO, sem humano no teclado: disparado por um evento (um push, uma issue, um cron). Os lugares mais comuns: CI/CD (GitHub Actions), um container/serverless acionado por webhook, ou um worker agendado. O segredo é o modo HEADLESS — o agente recebe a tarefa por argumento, executa e devolve o resultado, sem prompts interativos.

Onde hospedar um agente:

Ambiente	Disparado por	Bom para
GitHub Actions	push / PR / issue	Revisão de PR, fix de bugs, docs
Cron / scheduler	horário	Relatórios diários, triagem de issues
Serverless (Lambda)	webhook / fila	Responder eventos sob demanda
Container 24/7	fila / API	Bots interativos, alto volume

Modo headless — agente sem interação (script/CI):

```
# Claude Code em modo não-interativo: passa a tarefa e sai.
$ claude -p "Leia o erro em logs/erro.txt, corrija o código e rode os testes" \
  --output-format json

# Gemini CLI headless:
$ gemini -p "Gere o CHANGELOG a partir dos commits desde a última tag"

# Útil em qualquer pipeline: a saída (JSON/texto) vira input do próximo passo.
```

GitHub Actions — agente revisa toda PR automaticamente:

```
# .github/workflows/revisao-agente.yml
name: Revisao por Agente
on:
  pull_request:
    types: [opened, synchronize]

jobs:
  revisar:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      pull-requests: write      # para comentar na PR
    steps:
      - uses: actions/checkout@v4
        with: { fetch-depth: 0 }

      - name: Rodar agente de revisao
        env:
          ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
        run: |
          npm install -g @anthropic-ai/claude-code
          claude -p "Revise o diff desta PR (git diff origin/main).
            Aponte bugs, riscos de segurança e melhorias.
            Seja conciso: 1 linha por problema." \
            --output-format text > revisao.txt
          gh pr comment ${ github.event.number } --body-file revisao.txt
          # GH_TOKEN herdado automaticamente para o gh CLI
```

Segurança em CI — leia antes de dar a chave

Em CI o agente roda com acesso ao seu repo e a uma API key. Regras:

NUNCA exponha segredos em logs; use secrets do GitHub (nunca hardcoded).

Dê permissões MÍNIMAS (read quando possível). Cuidado com 'pull_request_target' em repos públicos — PRs de terceiros podem injetar comandos. Rode em sandbox e limite o que o agente pode tocar.

Guarda-corpos para agentes autônomos em produção:

- Orçamento de tokens/custo por execução — aborte se passar do limite
- Timeout e max de iterações — nada de loop infinito comendo créditos
- Lista branca de ferramentas — em CI, desligue o que não for necessário
- Tudo em git/branch — o agente nunca empurra direto na main
- Human-in-the-loop nos pontos críticos — aprovação antes de deploy/merge
- Logs e observabilidade — registre cada ação para auditar depois
- Idempotência — rodar de novo não deve duplicar efeitos

Observabilidade — o que medir:

Métrica	Por quê	Ferramenta
Tokens / custo	Não estourar orçamento	Logs próprios, LangSmith
Latência	UX e custo de runner	Métricas do CI
Taxa de sucesso	Saber se confia no agente	Evals automatizados
Ações executadas	Auditoria e segurança	Trace / logs estruturados

Guard-rail simples de custo (wrapper em Python):

```
import anthropic

LIMITE_TOKENS = 50_000          # teto por execução
gasto = 0

def chamar_com_teto(client, **kwargs):
    global gasto
    if gasto > LIMITE_TOKENS:
        raise RuntimeError(f"Orçamento estourado: {gasto} tokens")
    resp = client.messages.create(**kwargs)
    gasto += resp.usage.input_tokens + resp.usage.output_tokens
    print(f"[custo] +{resp.usage.output_tokens} | total={gasto}")
    return resp

# Em CI: se estourar, o job falha cedo em vez de consumir todos os créditos.
```

Quiz relâmpago

Qual é a maior diferença ao rodar um agente em CI vs no seu terminal?

☐ Em CI o modelo é mais inteligente

☒ Em CI ele roda headless (sem humano), então guarda-corpos e segredos são críticos

☐ Em CI não precisa de API key

☐ Em CI o agente não pode editar arquivos

 Sem humano no loop, qualquer erro vira automático. Por isso: permissões mínimas, limite de custo, sandbox e segredos via secrets — nunca hardcoded.

Avançado**🔗 Exercício 7 — Agente revisor na sua PR**

Crie um workflow do GitHub Actions que, a cada PR aberta, roda um agente em modo headless para revisar o diff e comentar na PR. Adicione um limite de custo e permissões mínimas. Abra uma PR de teste e veja o comentário automático aparecer.



Guarde a API key em Settings → Secrets → Actions (nunca no YAML)



Use 'permissions: pull-requests: write' só onde precisa comentar



Comece logando o diff num arquivo antes de mandar pro agente — depure barato



Teste o comando 'claude -p ...' localmente antes de jogar no CI

Próximos passos — sua jornada depois do curso:

- Automatize 1 tarefa chata sua esta semana (changelog, triagem de issues...)
- Versione um CLAUDE.md no seu projeto real e evolua a cada sessão
- Monte um workflow de revisão de PR com agente (Exercício 7)
- Experimente um MCP server útil para o seu contexto (banco, Jira, Notion)
- Quando dominar 1 agente, evolua para multi-agentes só onde fizer sentido
- Compartilhe seus PRDs e skills com o time — conhecimento que escala

Aprofundar em produção:

- **Claude Code em CI/CD** — Rodar o agente no GitHub Actions
<https://docs.anthropic.com/en/docs/claude-code/github-actions>
- **Claude Code Headless** — SDK e modo não-interativo
<https://docs.anthropic.com/en/docs/claude-code/sdk>
- **GitHub Actions Security** — Segredos e permissões em workflows
<https://docs.github.com/en/actions/security-guides>
- **LangSmith** — Observabilidade e evals de agentes
<https://docs.smith.langchain.com>